



[Home](#) > [Blog](#) > [ANDROID BOOT TIME OPTIMIZATION TOOLS ANALYSIS](#)

Android Boot Time Optimization – Tools and Analysis

Gopalakrishnan M

28. November 2024 Categories: Android, Linux, Technology,

In Android-based embedded systems and mobile devices, knowing how the boot process works is just the first step. Once familiar with the boot flow, the next step is mastering the tools that help analyze, debug, and optimize this sequence. Whether you're working on performance tuning, debugging startup issues, or validating system readiness in automotive or consumer electronics, tool-assisted insights are essential.

In this blog, we will focus from the boot sequence to a practical guide on the essential tools used to profile, trace, and analyze the Android boot process.

Understanding the Android Boot Process

The Android boot sequence is a complex composition of various components, each contributing to the overall startup time. A typical boot process encompasses the following stages:



1. **Boot ROM:** Loads the first stage of the bootloader into internal RAM.

3. **Kernel Initialization:** Sets up core system functions, initializes drivers, and mounts the root filesystem.

4. **init Process:** Executes initialization scripts (init.rc) to start system services.

5. **Zygote Startup:** Prepares the runtime environment for Android applications by preloading classes and resources.

6. **System Server Initialization:** Launches essential system services that constitute the Android framework.

7. **Home Screen Display:** Renders the user interface, signaling readiness for user interaction.

Each phase presents opportunities for optimization, and a granular analysis is essential to pinpoint areas that contribute most significantly to boot delays.

Measuring Boot Time

Accurate measurement is the basis of effective optimization. Several tools and techniques can be employed to profile and analyze the boot process.

Kernel printk Timestamps

A simple but effective method to analyze boot time in embedded Linux systems is by leveraging printk timestamps. This requires minimal setup and gives a quick view of where the time is spent during early boot.

Enable kernel printk timestamps in below approach for system already booted

1. `echo Y > /sys/module/printk/parameters/time`

Enable kernel printk timestamps in below approach from menuconfig

2. `CONFIG_PRINTK_TIME=y`

This setting ensures that each log entry is timestamped, facilitating precise measurement of kernel events.

Capture the logs via serial/UART interface. Use below command in a booted system to capture the log for analysis

3. *dmesg > boot.log*

Look for long gaps in timestamps, driver initialization delays, initcalls time consumption, waiting loops for hardware response etc to analyze the boot time.

initcall_debug

The `initcall_debug` kernel parameter provides detailed information about the execution time of each initialization function. Profiling them helps identify which drivers or components slow down the boot process.

Enable Initcall in below approach from menuconfig

4. *CONFIG_INITCALL_DEBUG=y*

This adds timestamp for every init call for example

calling xyz_driver_init+0x0/0x1000 @ 0

initcall xyz_driver_init+0x0/0x1000 returned 0 after 123456 usecs

calling → function start

returned → function end with time taken in microseconds (usecs)

Kernel organizes initcalls in levels:

- `early_initcall`
- `core_initcall`
- `subsys_initcall`
- `device_initcall`
- `late_initcall`

Use wisely to defer non-critical init to later stages.

Bootchart

Bootchart is a tool used to collect and visualizes data about the system's boot process, including CPU usage, disk I/O, and process execution.

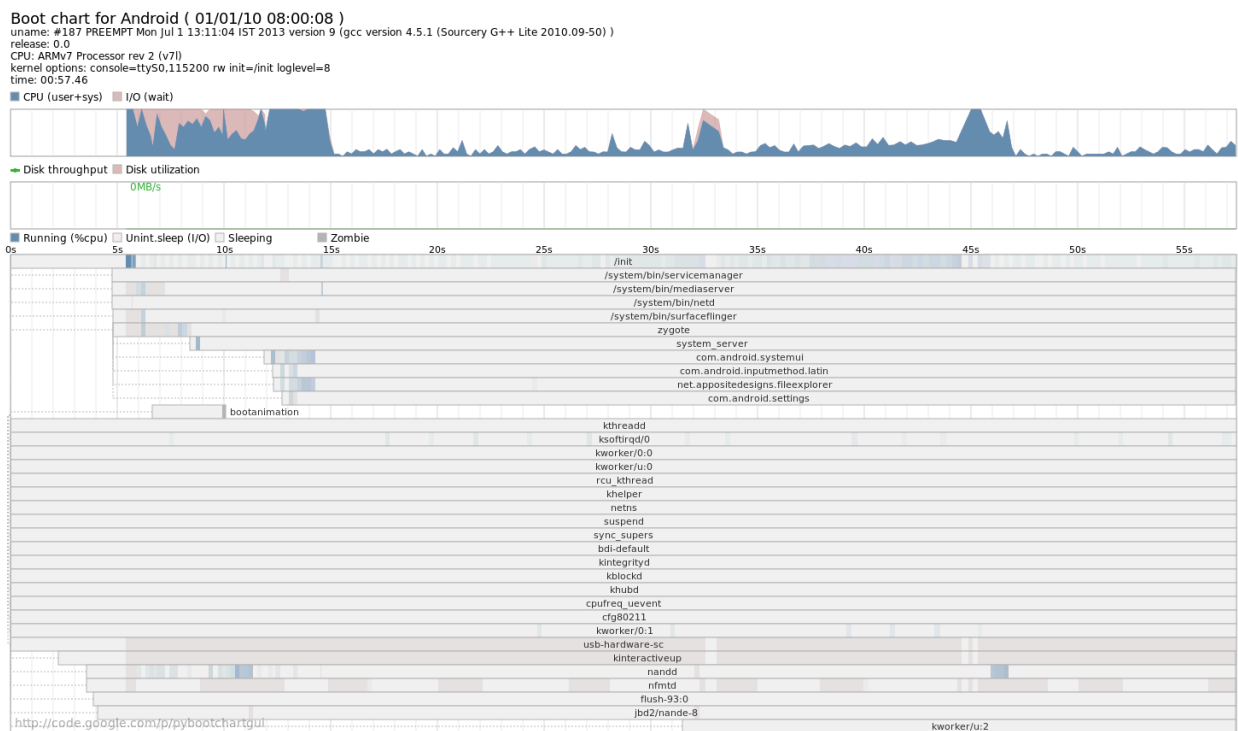
Enable Bootchart in one of the below approaches

1. For systems using systemd -> `systemd-analyze plot > bootchart.svg`
2. For init based system enable in kernel config ->
`CONFIG_BOOTCHART=y`
3. Another option is to execute a command from initramfs ->
`bootchartd start`

After device boots, Bootchart logs will be generated in `/var/log/bootchart.tgz`

By using bootchart tool in hostpc or pybootchartgui online tool, we can generate the bootchart image.

Ex : bootchart bootchart.tgz



Android BootChart

Bootchart provides a graphical representation of the boot sequence, helping in the identification of bottlenecks such as

Symptom**Cause**

Long gaps	Waiting for hardware/event
Serial process startup	No parallelism
High I/O wait	Filesystem or Storage bottleneck
Repeated retries	Driver or daemon errors

Ftrace

Ftrace is a powerful tracing framework built into the Linux kernel, allowing developers to monitor function calls and events during the boot process.

Enable Ftrace via kernel config

```
CONFIG_FUNCTION_TRACER=y
CONFIG_FUNCTION_GRAPH_TRACER=y
CONFIG_FTRACE=y
CONFIG_TRACING=y
CONFIG_BOOT_TRACER=y (optional)
```

Mount tracefs if not automounted

```
mount -t tracefs nodev /sys/kernel/tracing
```

Approach to enable function level tracing

1. Enable Ftrace via kernel config

```
CONFIG_FUNCTION_TRACER=y
CONFIG_FUNCTION_GRAPH_TRACER=y
CONFIG_FTRACE=y
CONFIG_TRACING=y
CONFIG_BOOT_TRACER=y (optional)
```

2. Mount tracefs if not auto mounte

```
mount -t tracefs nodev /sys/kernel/tracing
```

3. Enable function graph tracing:

```
echo                                function_graph
/sys/kernel/tracing/current_tracer >
```

4. Start tracing:

```
echo 1 > /sys/kernel/tracing/tracing_on
```

5. Stop tracing

```
echo 0 > /sys/kernel/tracing/tracing_on
```

6. After boot, analyze the trace log located at /sys/kernel/tracing/trace.

Ftrace provides granular insights into function execution times, facilitating the identification of performance bottlenecks such as long running functions, nested delays & serial execution.

Logcat Analysis

logcat captures Android logs from early boot to Launcher start. It helps trace delays in Android Framework, SystemServer, HAL, and App startup. It captures system logs, including timestamps for service startups and other critical events.

1. Capture logs during boot with adb

```
adb logcat -b all -d > boot_logcat.log
```

2. Review timestamps and durations of service initializations to spot anomalies.

Key timestamps to monitor are:

Init, Vold, SurfaceFlinger, Systemserver, Activity Manager & Launcher

Logcat analysis helps in identifying services that introduce significant delays during boot such as

Area	Checkpoint	Delay Cause
Mount	vold, fs_mgr	slow I/O, fsck
Display	SurfaceFlinger	GPU/Panel init
SystemServer	Service start logs	HAL/Service/Native delays
Launcher	ActivityManager	Package scan, Dexopt, App init

Identifying Bottlenecks

Once measurement tools are in place, the next step is to analyze the collected data to identify components that disproportionately affect boot time. Common problems include:

Unused Services

Unnecessary services that start during boot can consume resources and extend startup time.

- Review `init.rc` scripts for services that may not be required for the device's intended functionality.
- Disable or remove non-essential services to streamline the initialization process.

Inefficient Driver Initialization

Drivers that are slow to initialize or probe hardware can introduce delays.

- Enable `initcall_debug` to profile driver initialization times.
- Modify driver code to reduce initialization overhead, such as removing unnecessary waits or tests.
- Defer the loading of non-essential drivers until after the system is operational.

Conclusion

Boot time optimization in AOSP is a continuous and iterative process that demands a clear understanding of the boot flow and disciplined engineering practices. Developers must start by measuring each stage of the boot process accurately using tools like kernel logs, `initcall_debug`, `bootchart`, and `logcat` markers. Once bottlenecks are identified, optimizations should target both hardware and software layers.

At the kernel and driver level, disabling unused drivers, deferring non-critical initializations, and leveraging asynchronous probing can significantly reduce time spent during early boot stages. In the Android framework, trimming unnecessary system services, delaying non-essential background tasks, and optimizing init.rc scripts are effective strategies.

Ultimately, boot time is not just about speed but about smart sequencing – critical components should load early while non-essential processes can wait. A systematic measurement, data-driven analysis, and careful trade-offs enable developers to achieve faster, smoother boot experiences in Android-based devices across industries.

[« Previous](#) [Next »](#)

Related Blogs

MIPI-CSI CAMERA DRIVER DEVELOPMENT FOR NVIDIA JETSON PLATFORMS

In this blog, we will discuss in detail about the camera interface and data flow in Jetson Tegra platforms and typical configuration and setup of a MIPI CSI driver. For specifics, we will consider Jetson Nano and Onsemi OV5693 camera.

[Read More](#)

LOW CODE EMBEDDED SYSTEMS DEVELOPMENT WITH FLINT

In this blog, we will explore the scope for No-Code and Low-Code development for embedded systems and how Flint tool can help realize the same for this field.

[Read More](#)

OTA ARCHITECTURE FOR SCALABLE DESIGNS

Now a days in 2022 due to technology growth, a product is having multiple features/use cases and it

has been upgraded for bug fixes and new features in the interest of customer/end.

Read More

INSTRUMENT CLUSTER DESIGN FOR ELECTRIC VEHICLES WITH RENESAS RL78

In any vehicle, the instrument cluster forms a critical part as it is the face of the vehicle that reflects the current state.

Read More

Subscribe to our Blog

Email

mehmet.cihangir@yahoo.com

Name

Mehmet Cihangir

Phone Number

905354282882

Message

Hi, I am



Ben robot değilim

reCAPTCHA
Gizlilik - Şartlar

Submit

About us

As a leading product engineering and digital transformation services company with deep multi-domain expertise, Embien offers comprehensive solutions enabling customers develop better products faster.

Company

Homepage

About Us

Case Studies

Blog

Insights

Services

Product Engineering

Cloud and Apps

Data and Analytics

Quality Assurance

Technology Consulting

Products

Sparklet GUI Library

Flint IDE

RAPIDSEA Suite

TestBot

Skycase IoT Platform and Apps

Get In Touch

India | USA | Germany | Israel

sales@embien.com

+1 (847) 920-8533

+91 80563 33955

© Copyright 2025. All Rights Reserved. Embien Technologies India Pvt Ltd.

Connect With Us

